

CSA0613 - Design and Analysis of Algorithms

CO5-AT2-Pseudocode Writing Task (Algorithm Design)

Divyesh S P (192424147).

Hamiltonian Path Problem Using Backtracking

1. Aim

To implement the **Hamiltonian Path Problem using Backtracking** and determine whether a given graph contains a path that visits every vertex exactly once.

2. Problem Statement

Given a graph, find whether a **Hamiltonian Path** exists. A Hamiltonian Path visits every vertex of the graph exactly once.

3. Algorithm

1. Represent the graph using an adjacency matrix.
2. Select a vertex as the starting point.
3. Add an unvisited adjacent vertex to the path.
4. Continue until all vertices are visited.
5. If no valid vertex is available, backtrack and try another vertex.
6. If all vertices are visited exactly once, display the Hamiltonian Path.
7. If no possible path is found, display failure.

4. Input

The graph is represented using an adjacency matrix.

```
0 1 1 0
1 0 1 1
1 1 0 1
0 1 1 0
```

Here, 1 represents an edge and 0 represents no edge.

5. Python Code

```
def hamiltonian_path(graph):
    n = len(graph)
    path = [-1] * n
    visited = [False] * n

    def backtrack(pos):
        if pos == n:
            return True

        for v in range(n):
            if not visited[v] and graph[path[pos - 1]][v] == 1:
                path[pos] = v
                visited[v] = True

                if backtrack(pos + 1):
                    return True

                visited[v] = False
                path[pos] = -1

        return False

    for start in range(n):
        path[0] = start
        visited[start] = True
```

```
        if backtrack(1):
            return path

        visited[start] = False

    return None

graph = [
    [0, 1, 1, 0],
    [1, 0, 1, 1],
    [1, 1, 0, 1],
    [0, 1, 1, 0]
]

result = hamiltonian_path(graph)

if result:
    print("Hamiltonian Path exists:")
    print(" -> ".join(map(str, result)))
else:
    print("No Hamiltonian Path exists.")
```

6. Output

After executing the program:

Hamiltonian Path exists:

0 -> 1 -> 2 -> 3

Output:

```
# Input graph
graph = [
    [0, 1, 0, 0],
    [1, 0, 0, 0],
    [0, 0, 0, 1],
    [0, 0, 1, 0]
]

# Find Hamiltonian Path
result = hamiltonian_path(graph)

if result:
    print("Hamiltonian Path exists:")
    print(" -> ".join(map(str, result)))
else:
    print("No Hamiltonian Path exists.")
```

No Hamiltonian Path exists.

7. Test Case – No Hamiltonian Path

Input:

```
0 1 0 0
1 0 0 0
0 0 0 1
0 0 1 0
```

Output:

No Hamiltonian Path exists.

Output:

```
# Input graph
graph = [
    [0, 1, 0, 0],
    [1, 0, 0, 0],
    [0, 0, 0, 1],
    [0, 0, 1, 0]
]

# Find Hamiltonian Path
result = hamiltonian_path(graph)

if result:
    print("Hamiltonian Path exists:")
    print(" -> ".join(map(str, result)))
else:
    print("No Hamiltonian Path exists.")
```

No Hamiltonian Path exists.

8. Complexity

- **Time Complexity:** $O(n!)$
- **Space Complexity:** $O(n)$
- Backtracking reduces unnecessary exploration by abandoning invalid paths early.

9. Result

The **Hamiltonian Path Problem** was successfully implemented using the **Backtracking technique**. The program correctly identifies whether a Hamiltonian Path exists and displays the valid path when available. Invalid paths are eliminated using adjacency and visited-vertex constraints.